TagProtect CONFIRMED WORKING - Impinj Monza 4i on FXR90

TagProtect works. Proven end to end on genuine Impinj Gen2X tags, with a control tag and repeated trials.

This supersedes the inconclusive result in `../TAG_PROTECT_REPORT.md`. **The user was right**: the earlier tags were not Gen2X-capable, which is why nothing happened.

Reader	10.233.48.49 - FXR90, reader app 5.0.4
Antennas	5 and 6 (1-4 disconnected on this unit)
Power	30 dBm
Date	2026-09-07
Tags	2 x Impinj Monza 4i (MDID 0x801, TMN 0x1B0)
Password	12348765 - already present on both tags
Result	[OK] Protect, hide, visibility and unprotect all confirmed

---

1. Headline result

Phase	Gen2X applied	Target seen	Control seen
Baseline	none	5/5	5/5
After enableTagProtection	tagProtect	1/5	5/5
Gen2X cleared entirely	none	0/5	5/5
After enableTagVisibility	tagProtect	5/5	5/5
After disableTagProtection	none	5/5	5/5

Five 10-second inventories per phase. The control tag - `e28011b0a505007698ca0895`, an identical Monza 4i never named in any request - stayed visible **5/5 in every single phase**, which isolates the effect to the targeted EPC.

The third row is the decisive one. With **no Gen2X configuration loaded at all**, the target is still invisible 0/5. That proves the protection is written into the tag's silicon, not applied reader-side - the tag is genuinely RF-silent.

The single hit in row 2 run 1 was the first scan after applying the config; the protect operation had not yet reached the tag. Runs 2-5 are all misses.

---

2. Why the earlier attempt failed - the user's hypothesis was correct

The previous testing on `10.233.48.36` used whatever tags were in that field. Decoding their TIDs afterwards:

Tag used before	MDID	TMN	Chip	Gen2X?
e2806894000040017790e471	0x806	0x894	not Impinj	[X] never
41544035a880c80000123bea	0x801	0x130	Monza R6	[X] no Protected Mode
333311112222333344445555	0x801	0x1C1	Monza X-2K	? unverified

None was a confirmed Gen2X part. The tags used here:

Tag	MDID	TMN	Chip	Gen2X?
e28011b0a5050076c4d7530a	0x801	0x1B0	Monza 4i	[OK] yes
e28011b0a505007698ca0895	0x801	0x1B0	Monza 4i	[OK] yes

The API was never at fault. The tags were wrong. This closes the open item in the earlier report and resolves its headline question.

It also validates the guidance in Zebra's MAUI Gen2X tutorial:

> "Users should verify whether their tags support these operations by referring to Impinj Gen2X tag specifications"

Checking the chip is not optional - it was the entire difference between a working feature and an apparent defect.

---

3. The tags already had a password

Both tags read back with a **pre-programmed** access password:

```
GET RESERVED words 0-3  ->  "1234876512348765"
   kill   password (w0-1) = 12348765
   ACCESS password (w2-3) = 12348765
```

So the prerequisite was already satisfied and **no write was needed** - the existing password `12348765` was used throughout. Nothing was written to either tag's memory in this entire test run, and the password was re-verified unchanged at the end.

This is a cleaner test than writing a password first: it removes the write step as a variable entirely.

---

4. The exact sequence

4.1 Route tag data somewhere readable

The reader was publishing to a disconnected `WEBSOCKET_TEST` endpoint. Repointed to a local MQTT broker:

```
PUT /cloud/config
{"READER-GATEWAY":{"endpointConfig":{"data":{"event":{"connections":[{
  "name":"DATA_MQTT_49","description":"local broker","type":"mqtt",
  "additionalOptions":{"retention":{"maxEventRetentionTimeInMin":500,
                                    "maxNumEvents":150000,"throttle":100}},
  "options":{"enableSecurity":false,
             "endpoint":{"hostName":"10.117.229.18","port":1883,"protocol":"tcp"},
             "additional":{"cleanSession":true,"clientId":"fxr90-49-data",
                           "keepAlive":60,"qos":0,"debug":false,
                           "reconnectDelay":2,"reconnectDelayMax":10},
             "publishTopic":["fxr90-49/tevents"],"subscribeTopic":[]}}]}}}}}
```

-> 200, `DATA_MQTT_49 connected`

> **App 5.0.4 is stricter than 5.0.7 here.** Two simpler payloads were rejected with `invalid "additional" JSON object` - including one that omitted `additional` entirely. The `additionalOptions` wrapper had to be included to match the shape already stored on the reader. See §8.

4.2 Identify the chips

```
PUT /cloud/mode
{"type":"CUSTOM","antennas":[5,6],"transmitPower":[30,30],
 "accesses":[{"type":"READ","config":{"membank":"TID","wordPointer":0,"wordCount":6}}],
 "tagMetaData":["ANTENNA","RSSI"]}
```

```
PUT /cloud/start {}  ->  ... 14 s ...  ->  PUT /cloud/stop
```

EPC	Ant	RSSI	TID	MDID	TMN
e28011b0a5050076c4d7530a	5	?23	e28011b02000b30a26ba03b6	0x801	0x1B0
e28011b0a505007698ca0895	5	?37	e28011b02000a895c65003b4	0x801	0x1B0

Only two tags in the field - no other traffic to confuse the measurement.

4.3 Read the existing password

```
PUT /cloud/mode
{"type":"CUSTOM","antennas":[5,6],"transmitPower":[30,30],
 "filter":{"value":"e28011b0a5050076c4d7530a","match":"prefix","operation":"include"},
 "accesses":[{"type":"READ","config":{"membank":"RESERVED","wordPointer":0,"wordCount":4}}],
 "tagMetaData":["ANTENNA","RSSI"]}
```

Both tags -> `"1234876512348765"`.

4.4 Baseline visibility - 5 plain scans

```
PUT /cloud/mode
{"type":"CUSTOM","antennas":[5,6],"transmitPower":[30,30],"tagMetaData":["ANTENNA","RSSI"]}
```

```
PUT /cloud/start {}   x5, 10 s each
```

Target **5/5**, control **5/5**.

4.5 Protect the target

```
PUT /cloud/impinjGen2X
{"tagProtect":{"action":"enableTagProtection",
               "password":"12348765",
               "tagID":"e28011b0a5050076c4d7530a",
               "enableShortRange":false}}
```

-> 200, stored verbatim. Then five scans with the flag:

```
PUT /cloud/start {"applyImpinjGen2X":true}   x5, 10 s each
```

Target **1/5** (only run 1), control **5/5**.

4.6 The decisive check - clear Gen2X and rescan

```
PUT /cloud/impinjGen2X {"fastID":{"enabled":false}}
PUT /cloud/start {}                                  x5
```

Target **0/5**, control **5/5**.

No Gen2X config, no flag - and the tag is still gone. Protection lives on the tag.

4.7 Read the protected tag with the password

```
PUT /cloud/impinjGen2X
{"tagProtect":{"action":"enableTagVisibility","password":"12348765"}}

PUT /cloud/start {"applyImpinjGen2X":true}   x5
```

Target **5/5**, control 5/5.

This is the mechanism working exactly as documented - a protected tag is invisible to everyone except a reader that supplies the correct password.

4.8 Unprotect

```
PUT /cloud/impinjGen2X
{"tagProtect":{"action":"disableTagProtection",
               "password":"12348765",
               "tagID":"e28011b0a5050076c4d7530a"}}

PUT /cloud/start {"applyImpinjGen2X":true}
```

Then Gen2X cleared and five plain scans: target **5/5**, control 5/5.

Fully reversible, and the tag is back to its original state.

---

5. What this confirms

Claim	Status
enableTagProtection makes a Gen2X tag RF-silent	[OK] confirmed - 5/5 -> 0/5
Protection is written to the tag, not the reader	[OK] confirmed - persists with no Gen2X loaded
enableTagVisibility lets an authorised reader read it	[OK] confirmed - 0/5 -> 5/5
disableTagProtection reverses it	[OK] confirmed - back to 5/5
The effect is specific to the targeted EPC	[OK] confirmed - control 5/5 in all five phases
The two-step configure-then-apply flow is required	[OK] confirmed
Requires a Gen2X-capable Impinj chip	[OK] confirmed - this was the missing piece

---

6. Corrections to the earlier report

`../TAG_PROTECT_REPORT.md` §3.3 recorded "no observable effect, cause not isolated" and listed two outstanding checks. Both are now resolved:

Earlier open item	Resolution
Test A - verify the chip is Gen2X-capable	[OK] This was the cause. Monza 4i works; the earlier tags were non-Impinj, Monza R6, or unverified X-2K
Test B - read from a second reader that lacks the password	[OK] Effectively answered. §4.6 clearing Gen2X entirely gives the same evidence - an unauthorised read attempt - and the tag is invisible

The earlier report's caution was correct: it concluded "cause not isolated" rather than declaring a firmware defect. Had it claimed a bug, that claim would now be wrong.

No API or firmware defect exists in TagProtect. The feature works as documented on appropriate hardware.

---

7. Tag and reader state after testing

	Before	After	
Target visibility	5/5	5/5	[OK] restored
Control visibility	5/5	5/5	[OK] untouched
Target access password	12348765	12348765	[OK] unchanged
Target kill password	12348765	12348765	[OK] unchanged
impinjGen2X config	(empty)	{"fastID":{"enabled":false}}	idle
Reader mode	CUSTOM, ant [1,5,6], 17 dBm	restored	[OK]
Data endpoint	WEBSOCKET_TEST	restored	[OK]

Nothing was written to either tag's memory at any point - the existing password was read and used, never modified. Verified by reading RESERVED words 0-3 again at the end.

Both tags are left unprotected and fully readable.

---

8. Differences found between app 5.0.4 and 5.0.7

Incidental, but useful - this was the first test on 5.0.4:

	5.0.7 (10.233.48.36)	5.0.4 (10.233.48.49)
Model	FXR60	FXR90
data.event connection shape	options.additional accepted alone	requires additionalOptions wrapper too
Rejected payloads	-	2 attempts rejected: invalid "additional" JSON object
Antennas connected	1	5 and 6
transmitPower	scalar 30 accepted	array [30,30] per antenna

The rejection message is misleading in the same way seen elsewhere: it names `additional` even when the payload **contains no `additional` key at all**. The actual requirement was the sibling `additionalOptions` object.

Practical guidance: `GET /cloud/config` first and match the shape already stored on that reader, rather than assuming a payload that worked on another unit will be accepted.

---

9. Evidence

```
tagprotect-CONFIRMED-monza4i/
|-- TAGPROTECT_CONFIRMED.md   <- this file
`-- evidence/                  30 raw MQTT capture files
    |-- tid.ndjson             the TID sweep that identified the chips
    |-- res_tgt.ndjson         target RESERVED read (password 12348765)
    |-- res_ctl.ndjson         control RESERVED read
    |-- base_1..5.ndjson       baseline, 5/5 visible
    |-- prot_1..5.ndjson       protected, 1/5 visible
    |-- after_1..5.ndjson      Gen2X cleared, 0/5 visible  <- decisive
    |-- vis_1..5.ndjson        enableTagVisibility, 5/5 visible
    |-- unprot_apply.ndjson    unprotect being applied
    |-- final_1..5.ndjson      after unprotect, 5/5 visible
    `-- res_final.ndjson       password re-verified unchanged
```

Each `.ndjson` is newline-delimited tag events straight off `mqtt://10.117.229.18:1883` topic `fxr90-49/tevents`. Count target hits with:

```
grep -c e28011b0a5050076c4d7530a evidence/base_1.ndjson
```

10. Related

File	Contents
../TAG_PROTECT_HOWTO.md	The shareable procedure - update §7 with this result
../TAG_PROTECT_REPORT.md	The earlier inconclusive run and its method critique
../WHAT_WE_DID.md	Chronological record of that earlier run
../v3-all-examples/GEN2X_ALL_EXAMPLES_REPORT.md	All 12 Gen2X examples from spec v3.0.0